A Stack Machine

Ads by Google
Stop seeing this ad Why this ad?

The S-machine* is a stack machine organized to simplify the implementation of block structured languages. It provides dynamic storage allocation through a stack of activation records. The activation records are linked to provide support for static scoping and they contain the context information to support procedures.

Machine Organization

The S-machine consists of two stores, a program store, **C** (organized as an array and is read only), and a data store, **S** (organized as a stack). There are four registers, an instruction register, **IR**, which contains the instruction which is being interpreted, the stack top register, **T,** which contains the address of the top element of the stack, the program address register, **PC**, which contains the address of the next instruction to be fetched for interpretation, and the current activation record register, **AR**, which contains the base address of the activation record of the procedure which is being interpreted. Each location of **C** is capable of holding an instruction. Each location of **S** is capable of holding an address or an integer. Each instruction consists of three fields an operation code and two parameters.

Storage	*Registers*	*Instruction*
C - code **S** - stack	**IR** - instruction register **T** - stack top pointer **PC** - program counter **AR** - activation record pointer	OpCode, arg_1, arg_2

Instruction Set

S-codes are the machine language of the S-machine. S-codes occupy four bytes each. The first byte is the operation code (op-code). There are nine basic S-code instructions, each with a different op-code. The second byte of the S-code instruction contains either 0 or a lexical level offset, or a condition code for the conditional jump instruction. The last two bytes taken as a 16-bit integer form an operand which is a literal value, or a variable offset from a base in the stack, or a S-code instruction location, or an operation number, or a special routine number, depending on the op-code.
The action of each instruction is described using a mixture of English language description and mathematical formalism. The mathematical formalism is used to note changes in values that occur to the registers and the stack of the S-machine. Data access and storage instructions require an offset within the activation record and the level difference between the referencing level and the definition level. Procedure calls require a code address and the level difference between the referencing level and the definition level.

Instruction	Operands	Comments
LIT	0,N	Load literal value N onto stack: T := T+ 1; S(T) := N
OPR	0,0	Return (from subroutine call)
	0,1	Negate: S(T) := -1*S(T)
	0,2	Add: S(T-1) := S(T-1) + S(T); T := T-1
	0,3	Subtract: S(T-1) := S(T-1) - S(T); T := T-1
	0,4	Multiply: S(T-1) := S(T-1) * S(T); T := T-1
	0,5	Divide: S(T-1) := S(T-1) / S(T); T := T-1
	0,6	undefined
	0,7	Mod: S(T-1) := S(T-1) mod S(T); T := T-1
	0,8	Equal: S(T-1) := if S(T-1) = S(T) then 1 else 0; T:= T-1
	0,9	Not equal: S(T-1) := if S(T-1) <> S(T) then 1 else 0; T:= T-1
	0,10	Less than: S(T-1) := if S(T-1) < S(T) then 1 else 0; T:= T-1
	0,11	Greater than or equal: S(T-1) := if S(T-1) >= S(T) then 1 else 0; T:= T-1
	0,12	Greater than: S(T-1) := if S(T-1) > S(T) then 1 else 0; T:= T-1
	0,13	Less than or equal: S(T-1) := if S(T-1) <= S(T) then 1 else 0; T:= T-1
	0,14	Or: S(T-1) := if(S(T-1) + S(T) > 1 then 1 else 0; T := T-1

	0,21	Copy: S(T+1):= S(T); T := T+1
Data Access Operations		
LOD	L,N	Load value of variable at level offset L, base offset N in stack onto top of stack T:= T + 1; S(T):= S(f(L,AR)+N)+3
LOD	255,0	Load byte from memory address which is on top of stack onto top of stack: S(T) := S(S(T))
LODX	L,D	Indexed load: S(T) := S(f(L,AR)+D+S(T)
STO	L,N	Store value on top of stack into variable location at level offset L, base offset N in stack: S(f(L,AR)+N+3):= S(T); T:= T - 1
STO	255,0	Store: S(S(T-1)) := S(T); T:=T-2
STOX	L,D	Indexed store: POP index, POP A, store A at (base of level offset L)+D+index
Control Operations		
CAL	L, N	call PROC or FUNC at S-code location N declared at level offset L: S(T+1):= f(ld,AR); {Static Link} S(T+2):= AR; {Dynamic Link} S(T+3):= P; {Return Address} AR:= T+1; {Activation Record} PC:= N; {Program Counter} T:= T+3 {Stack Top}
JMP	0, N	JUMP: P := N;
JPC	C, N	JUMP: if S(T) = C then P:= N; T:= T-1
CSP	0, 0	CHARACTER Input: T := T+1, S(T) := input;
	0, 1	CHARACTER Output: Output := S(T); T := T-1;
	0, 2	INTEGER Input: T := T+1; S(T) := input
	0, 3	INTEGER Output: Output := S(T); T := T-1
	0, 8	STRING Output: L := S(T); T := T-1; FOR I := 1 to L DO BEGIN Output := S(T); T := T-1 END

Where the static level difference between the current procedure and the called procedure is ld.
os is the offset within the activation record, ld is the static level difference between the current activation record and the activation record in which the value is to be stored and
f(ld,a) = if i=0 then a else f(i-1,S(a))

Operation

The registers and the stack of the S-machine are initialized as follows:

```
P := 0; {Program Counter}
AR := 0; {Activation Record}
T := 2; {Stack Top}
S[0] := 0; {Static Link}
S[1] := 0; {Static Dynamic Link}
S[2] := 0; {Return Address}
```

The machine repeatedly fetches the instruction at the address in the register P, increments the register P and executes the instruction until the register P contains a zero.

```
execution-loop: I := C(P);
                P := P+1;
                interpret(I);
                if P > 0 -> execution-loop
```

The Stack Machine Module

The implementation of the stack machine is straight forward. The instruction set and the structure of an instruction are defined as follows:

```
/* OPERATIONS: Internal Representation */
enum code_ops { HALT, STORE, JMP_FALSE, GOTO, DATA, LD_INT,
                LD_VAR, READ_INT, WRITE_INT, LT, EQ, GT,
                ADD, SUB, MULT, DIV, PWR };

/* OPERATIONS: External Representation */
char *op_name[] = {"halt", "store", "jmp_false", "goto",
                   "data", "ld_int", "ld_var", "in_int",
                   "out_int", "lt", "eq", "gt",
                   "add", "sub", "mult", "div", "pwr" };

struct instruction { enum code_ops op; int arg; };
```

The definitions of the registers, the program counter {\tt pc}, the instruction register {\tt ir}, the activation record pointer {\tt ar} (which points to be begining of the current activation record), and the pointer to the top of the stack {\tt top}, are straight forward.

```
int pc = 0;
truct instruction ir;
int ar = 0;
int top = 0;
```

The fetch-execute cycle repeats until a halt instruction is encountered.

```
void fetch_execute_cycle()
{ do { /* Fetch */ ir = code[pc++];
       /* Decode & Execute */
       switch (ir.op) {
          case HALT      : printf( "halt\n" ); break;

          case READ_INT  : printf( "Input: " );
                             scanf( "%ld", &stack[ar+ir.arg] );     break;
          case WRITE_INT : printf( "Output: %d\n", stack[top--] );  break;
          case STORE     : stack[ir.arg] = stack[top--];            break;
          case JMP_FALSE : if ( stack[top--] == 0 ) pc = ir.arg;
                                                                    break;
          case GOTO      : pc = ir.arg;                             break;
          case DATA      : top = top + ir.arg;                      break;
          case LD_INT    : stack[++top] = ir.arg;                   break;
          case LD_VAR    : stack[++top] = stack[ar+ir.arg];         break;
          case LT        : if ( stack[top-1] < stack[top] )
                                stack[--top] = 1;
                             else stack[--top] = 0;                 break;
          case EQ   : if ( stack[top-1] == stack[top] ) stack[--top] = 1; else stack[--top] = 0;        break;
          case GT   : if ( stack[top-1] > stack[top] )  stack[--top] = 1; else stack[--top] = 0; top--; break;
          case ADD  : stack[top-1] = stack[top-1] + stack[top]; top--; break;
          case SUB  : stack[top-1] = stack[top-1] - stack[top]; top--; break;
          case MULT : stack[top-1] = stack[top-1] * stack[top]; top--; break;
          case DIV  : stack[top-1] = stack[top-1] / stack[top]; top--; break;
          case PWR  : stack[top-1] = stack[top-1] * stack[top]; top--; break;
          default   : printf( "%sInternal Error: Memory Dump\n" ); break; } }
  while (ir.op != HALT); }
```

---

*This is an adaptation of: Niklaus Wirth, *Algorithms + Data Structures = Programs* Prentice-Hall, Englewood Cliffs, N.J., 1976.

See also Wilhelm and Maurer, *Compiler Design* Addison-Wesley 1995 pp. 7-60.